

ScopeLens

An evidence-first requirement change workbench

A complete explanation for an internship interview panel: the product problem, architecture, design choices, testing evidence, limitations, deployment model, and answers to likely questions.

Live demonstration

<https://scopelens-three.vercel.app>

Public source repository: <https://github.com/mr-kumuditha/scopelens>

Prepared for

Kumuditha Tharinda Liyanage | HND Software Engineering internship portfolio

What this project shows

- Product thinking: turn an ambiguous requirement change into a reviewable decision.
- Engineering discipline: constraints, transactions, lifecycle states, and explicit boundaries.
- Communication: an interface and handbook that a non-technical reviewer can follow.

Important live-demo note

Current Vercel mode

The public site uses fictional seed data and an in-memory serverless demonstration database. It is useful for viewing the interface and health status, but it is not a durable shared workspace. Local ScopeLens persists using PGlite; a hosted handover requires the configured external PostgreSQL adapter.

1. The problem and product idea

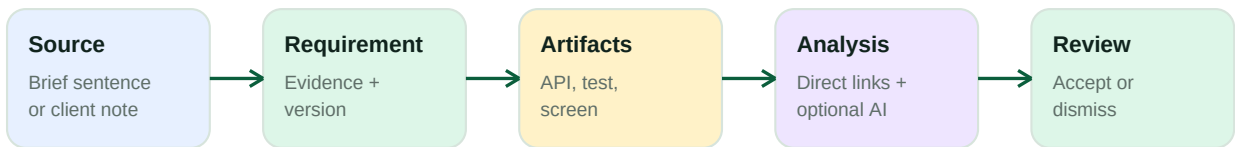
Small changes become expensive when teams lose the connection between a source statement, the requirement it created, and the implementation work it could affect. A booking rule can touch a screen, an API, an automated test, and a support process. Spreadsheets and chat messages often preserve only part of that story.

ScopeLens gives one owner a compact workspace to record the source evidence, version a requirement, connect work artifacts with exact evidence, compare versions, and make a human review decision.

The core outcome

*A change should end as a clear answer: **what changed, what might be affected, what evidence supports it, and who reviewed it.***

End-to-end product flow

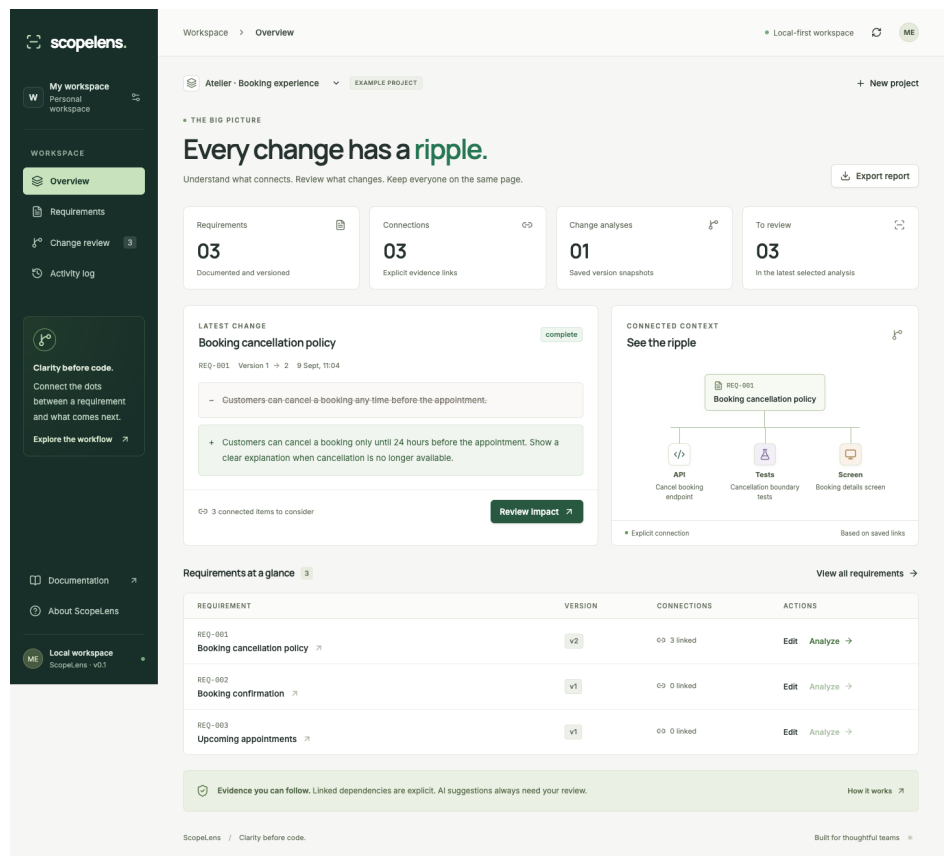


Every decision remains traceable to the saved source and the reviewed requirement version.

Actor	Need	ScopeLens response
Product owner	Keep the original intent	Stores a literal source excerpt from the project brief.
Developer	See affected work before coding	Links APIs, tests, screens, and requirement artifacts.
Reviewer	Avoid approving old analysis	Locks review if the requirement version has changed.
Interview panel	Assess real engineering thinking	Shows validation, limits, and architecture decisions.

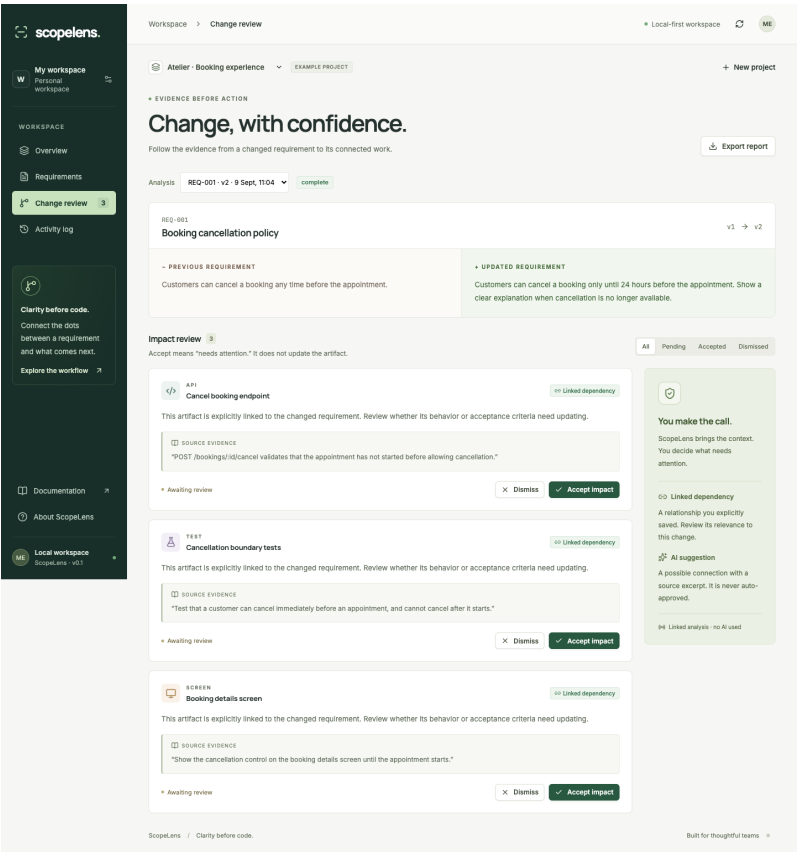
2. Experience and interface

The interface is deliberately calm and compact: a clear project overview, a requirements workspace, evidence connections, and a review view. The goal is to reduce cognitive load while keeping technical evidence visible when the user needs it.

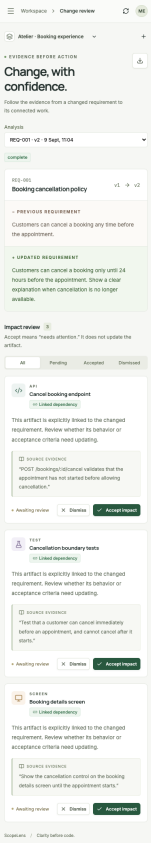


Overview: fictional Atelier booking example. It visualizes project context and the current change workbench.

Review flow



Change review: before/after requirement text, linked evidence, impact status, and a human decision.

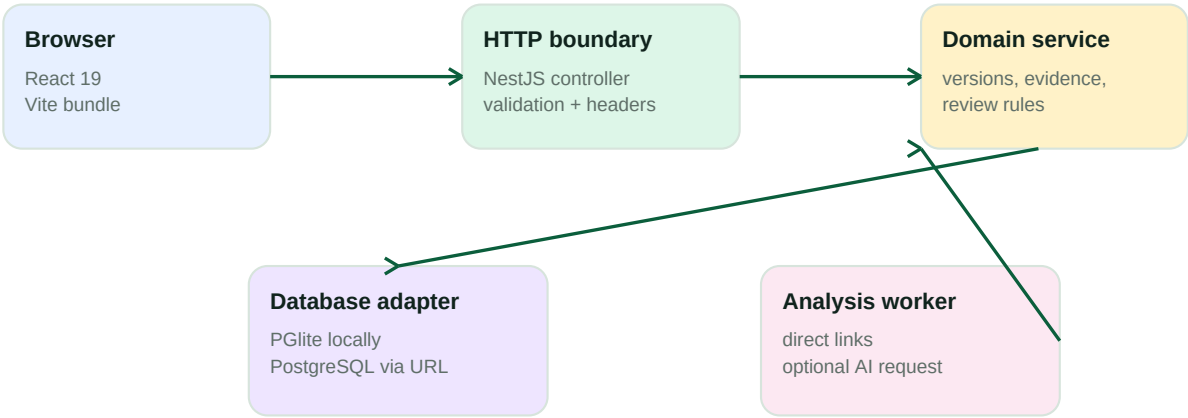


Mobile review: the same work can be inspected at phone width without document-level horizontal overflow.

3. Architecture foundation

ScopeLens is intentionally a modular monolith. It has one deployable backend and a compact domain service, while its responsibilities are still separated conceptually: HTTP boundary, validation, domain rules, storage, analysis, and review. That choice keeps the first release easy to reason about and test.

Modular-monolith foundation: one deployable service with explicit feature responsibilities.



Why this foundation is appropriate

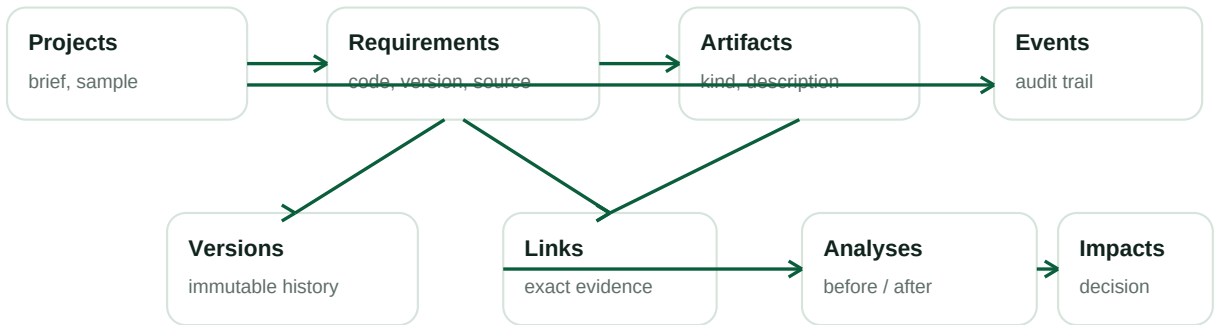
Decision	Why it helps an internship project	Trade-off
React + TypeScript	Shows component-based UI and static type safety.	Needs disciplined state and API contracts.
NestJS controller + service	Separates web concerns from business rules.	Not split into independent services yet.
PGlite local / PostgreSQL external	Runs easily locally while teaching relational constraints.	External database needs its own deployment validation.
Optional AI	Shows bounded integration, not blind automation.	Model quality needs a labelled evaluation set.

Request boundary

- The browser calls a same-origin JSON API.
- The controller validates input and applies security headers.
- The service owns rules for versioning, evidence, analysis, and review.
- SQL uses parameters; user input is never interpolated as an identifier.

4. Data model and change lifecycle

The data model does more than store screens. It encodes the relationships that make a change reviewable: a project owns requirements, artifacts, analysis records, and audit events; a requirement keeps its current state and immutable versions.



Data integrity rules

- Each requirement receives a project-scoped code such as REQ-001.
- A source excerpt must be an exact substring of the saved brief.
- A requirement edit requires the version the user read. A mismatch returns HTTP 409 instead of overwriting another change.
- A link must stay inside one project, be unique, and quote exact evidence from its selected artifact.
- An analysis stores the compared versions and snapshots its impacts. Later edits make it stale.
- A review decision is blocked when the analysis is stale or still running.

Change lifecycle

Step	Stored evidence	Protection
1. Capture	Brief and source excerpt	Exact-match source validation
2. Version	Current requirement and immutable version row	Expected-version check
3. Connect	Artifact description and evidence quote	Project scope and duplicate checks
4. Analyze	Before/after text and impact snapshot	One active job; explicit state
5. Review	Accepted or dismissed decision + event	Reject stale analysis server-side

5. What happens when a requirement changes

Example: an appointment cancellation rule changes from 'any time before the appointment' to 'up to 24 hours before'. ScopeLens does not automatically rewrite a test or API. It makes the relevant work visible, preserves the reason for the change, and asks a human to decide.

Detailed flow

Moment	System behaviour	Why it matters
Save edit	Validates title, body, reason, and expected version. Stores a new immutable version.	Preserves history and prevents lost updates.
Start analysis	Captures the prior/current text and known direct links.	Evidence does not disappear if an optional model fails.
Optional model	Receives bounded context; returned suggestions need a valid artifact and literal quote.	Avoids accepting invented citations.
Review	User accepts or dismisses the impact.	A decision is explicit, not inferred from a click elsewhere.
Edit again	Earlier analysis is marked stale and decisions are locked.	Prevents a reviewer from approving an old comparison.

The human boundary

AI can suggest candidates only when the owner explicitly requests it. Direct evidence links work without AI. ScopeLens never claims that a source quote makes a model suggestion correct.

Failure behaviour

- Invalid fields return 400 with a useful message.
- Missing or wrong-project records return 404.
- Competing edits, duplicate links, or outdated review return 409.
- A provider failure marks analysis failed but preserves direct evidence impacts.
- Unfinished jobs are marked failed after a process restart; the owner may explicitly rerun them.

6. Technology and resources

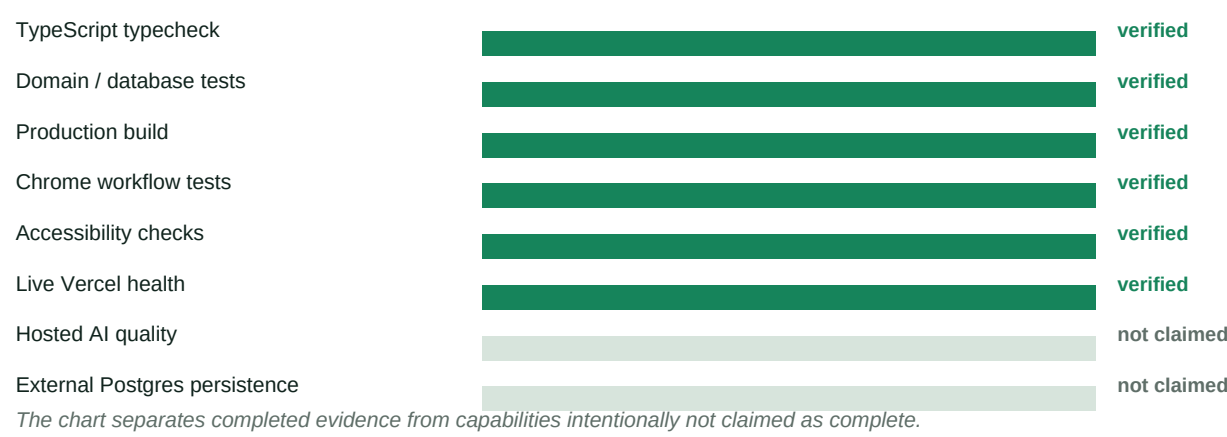
Area	Technology	Purpose in this project
Frontend	React 19, TypeScript, Vite	Fast single-page interface with typed components and bundled assets.
User interface	CSS design tokens, Lucide icons, Inter / Manrope / IBM Plex Mono	Readable hierarchy, responsive layout, locally bundled visual assets.
Backend	NestJS 12, Express 5, Zod	HTTP routing, validation, structured error boundary.
Data	PGlite, PostgreSQL adapter, pg	Local relational persistence with an external PostgreSQL path.
Quality	Vitest, Playwright, Axe, Prettier	Domain rules, browser workflows, accessibility checks, formatting.
Delivery	GitHub Actions, Vercel	Repeatable checks and live static/serverless deployment.
Optional intelligence	Hosted model adapter	Bounded impact suggestions with schema and exact-quote validation.

Useful resources for a reviewer

Resource	What to read
README.md	Product summary, setup, and current deployment boundary.
docs/ARCHITECTURE.md	System boundary, data model, transactions, and scaling limits.
docs/DECISIONS.md	Six architecture decision records and their trade-offs.
docs/API.md	Routes, request shapes, errors, and snapshot contract.
docs/TESTING.md	What automated checks prove and what they do not prove.
docs/INTERVIEW_GUIDE.md	Short demo flow and CV wording.
docs/DEPLOYMENT.md	Local, Docker, and Vercel deployment information.

7. Quality evidence

The project treats validation as evidence. A passing test is not used to claim an unrelated result. For example, structural tests validate AI response boundaries; they do not prove model accuracy.



Completed checks

Check	Evidence
Type safety	TypeScript typecheck passed.
Domain behaviour	13 Vitest checks passed using isolated in-memory PGlite databases and real SQL constraints.
Build	Vite frontend and compiled NestJS backend build passed.
Browser workflow	4 Chrome Playwright workflows passed in prior validation: project creation, evidence, versioning, analysis, review, reload, and responsive sample.
Accessibility	Axe A/AA checks passed in tested overview, review, requirements, activity, modal states; phone overflow was checked at 390 px.
Hosted deployment	Vercel production health endpoint returned 200 with demoMode true on 9 September 2026.
Dependencies	Dependency audit reported zero known vulnerabilities at the recorded check time.

What is intentionally not claimed

- Hosted AI accuracy or live provider performance: no provider key or labelled evaluation dataset is configured.
- Durable external PostgreSQL production persistence: adapter exists, but current Vercel site is the resettable demo configuration.
- Enterprise security, user roles, multi-tenancy, disaster recovery, or a complete assistive-technology audit.

8. Deployment and handover

There are two valid running modes, and they serve different purposes.

Mode	Storage	Best use	Important limit
Local application	PGlite in .data/scopelens or DATABASE_URL	Development, portfolio demo, a single owner workspace	Run one API process; process-local job coordination.
Current Vercel public demo	In-memory serverless demonstration data	Visual review of the interface and fictional sample	No durable shared data. Never enter real client or personal data.
Hosted handover target	External PostgreSQL via DATABASE_URL	A real private owner workspace	Needs database provision, token configuration, backups, and operational validation.

Local start

Commands

1. npm install
2. npm run dev
3. Open <http://127.0.0.1:5173>
4. Run npm run check before a handover.

Public link

Live Vercel deployment

<https://scopelens-three.vercel.app>

The linked production deployment is publicly reachable. Its health endpoint reports demoMode true. This is an honest portfolio demonstration, not a persistent customer data service.

What a production handover should add

- Provision an external PostgreSQL service and set DATABASE_URL in Vercel.
- Set a strong API_TOKEN (at least 32 characters) and keep it out of the repository.
- Add user authentication and authorization before sharing across people.
- Set backup, restore, retention, observability, and incident responsibilities.
- Replace the process-local analysis queue with database-backed job claiming before replicas.

9. Interview panel answers

Use these answers as a guide. Keep them conversational and connect each answer to a screen or code path where possible.

1. What problem does ScopeLens solve?

It makes requirement changes reviewable. It keeps the original source, versions the requirement, records affected artifacts with evidence, and requires a human decision before the work is treated as reviewed.

2. Why did you choose this project?

It is small enough to finish well, but it demonstrates product thinking, relational data modelling, API design, concurrency, lifecycle states, testing, accessibility, and deployment boundaries.

3. Why use a modular monolith instead of microservices?

There is one owner and one small workflow. A monolith keeps transactions and debugging simple. I separated responsibilities in the code so the application can split by feature only when the product needs that complexity.

4. How do you avoid two people overwriting the same requirement?

The client sends the version it read. The server checks it inside the edit transaction. If it differs from the stored version, the API returns HTTP 409 and the user must refresh before editing again.

5. What is immutable in the data model?

Requirement versions and analysis snapshots are preserved. The current requirement changes, but the previous body, reason, and version remain available for audit and comparison.

6. How do you prove a requirement came from the brief?

A source excerpt must be a literal substring of the saved project brief. This preserves provenance, but it does not claim that the requirement is semantically perfect.

7. Why does artifact evidence require an exact quote?

It creates a checkable connection. A reviewer can see the precise part of a test, API description, or screen note that supports the impact. It limits invented citations.

8. Does optional AI decide what needs changing?

No. AI is only an explicit suggestion path. Direct user-recorded links are collected deterministically. AI suggestions need schema validation and exact evidence; the user still accepts or dismisses each impact.

9. What happens when the model provider fails?

The analysis record becomes failed and preserves direct-link impacts. This protects the evidence-based workflow from an external provider failure.

10. Why can an old review not be accepted?

A review belongs to one requirement version. If the requirement changes again, the prior analysis no longer describes the current state, so the server blocks a decision until a new comparison is run.

10. More interview answers and next steps

11. Why PGlite locally?

It gives the project a real relational SQL engine without making a reviewer install Docker or a separate database. The same domain rules can use an external PostgreSQL adapter when deployed.

12. Can the project scale horizontally today?

No. The initial release has process-local serialization and job coordination. Horizontal scale needs atomic job claims in the database, per-transaction connections, migrations, and distributed concurrency tests.

13. What security exists today?

The server uses input validation, parameterized SQL, security headers, same-origin checks, and a strong owner token for hosted mode. It is not a multi-user enterprise security design; roles and identity are explicitly future scope.

14. Why is the Vercel site a demo instead of a full shared workspace?

Serverless instances do not share an in-memory database. I exposed the interface with fictional sample data for a safe public portfolio demonstration. A full handover needs external PostgreSQL and private access configuration.

15. How would you evaluate the AI feature?

I would build a human-labelled set of changes and expected affected artifacts, including no-impact and adversarial cases. I would report precision, recall, valid-evidence rate, latency, cost, model, prompt version, sample size, and date.

16. What did testing prove?

The domain tests prove specific constraints and lifecycle rules; browser checks prove selected user journeys and accessibility states. They do not prove hosted database operations, model quality, or every device and browser.

17. What would you build next?

First, durable hosted PostgreSQL and private authentication. Next, database-backed analysis jobs, artifact import integrations, team roles, a labelled AI evaluation harness, and backup/restore verification.

18. Give a 60-second project introduction.

ScopeLens is a requirement-change workbench. When a client changes a rule, developers need to know what work might be affected and why. I store the original source, version each requirement, connect APIs, tests, or screens with exact evidence, compare the old and new requirement, and require a human review decision. I chose a modular monolith with React, NestJS, and PostgreSQL concepts because it keeps the core transaction rules understandable. The public Vercel site is a safe fictional demo; a real shared workspace would use external PostgreSQL and authentication.

CV-ready description

Use after describing the project accurately

Built ScopeLens, a React/TypeScript and NestJS requirement-change workbench with PostgreSQL-compatible persistence, evidence-linked artifacts, immutable requirement history, and human impact review. Implemented optimistic concurrency, stale-analysis protection, database-backed tests, responsive UI, and a public Vercel demonstration.

Closing note

The strongest part of this project is not its size. It is the care in its boundaries: it identifies what the system can prove, preserves evidence for decisions, and states clearly what must be added before real user data is trusted to a hosted environment.